

程序设计实习（实验班-2026春）

课程总结、期末安排

授课教师：姜少峰

助教：方心童、葛致远、李雷思问、刘鹏飞

Email: shaofeng.jiang@pku.edu.cn

关于期末考试

时间地点

- 上机考试，闭卷，3小时
- 6月26日下午 x点xx - x点xx **具体时间地点会在教学网和答疑群通知，请关注！**
- 地点 理科一号楼1235，与上机位置相同（xx-xx，xx-xx号机器）
- 考试继续采用openjudge平台，但会切断internet（只有内网）
- 注意：（在心里）记好自己的openjudge账号密码！
- 建议考试前若干天去测试一次环境

题目构成

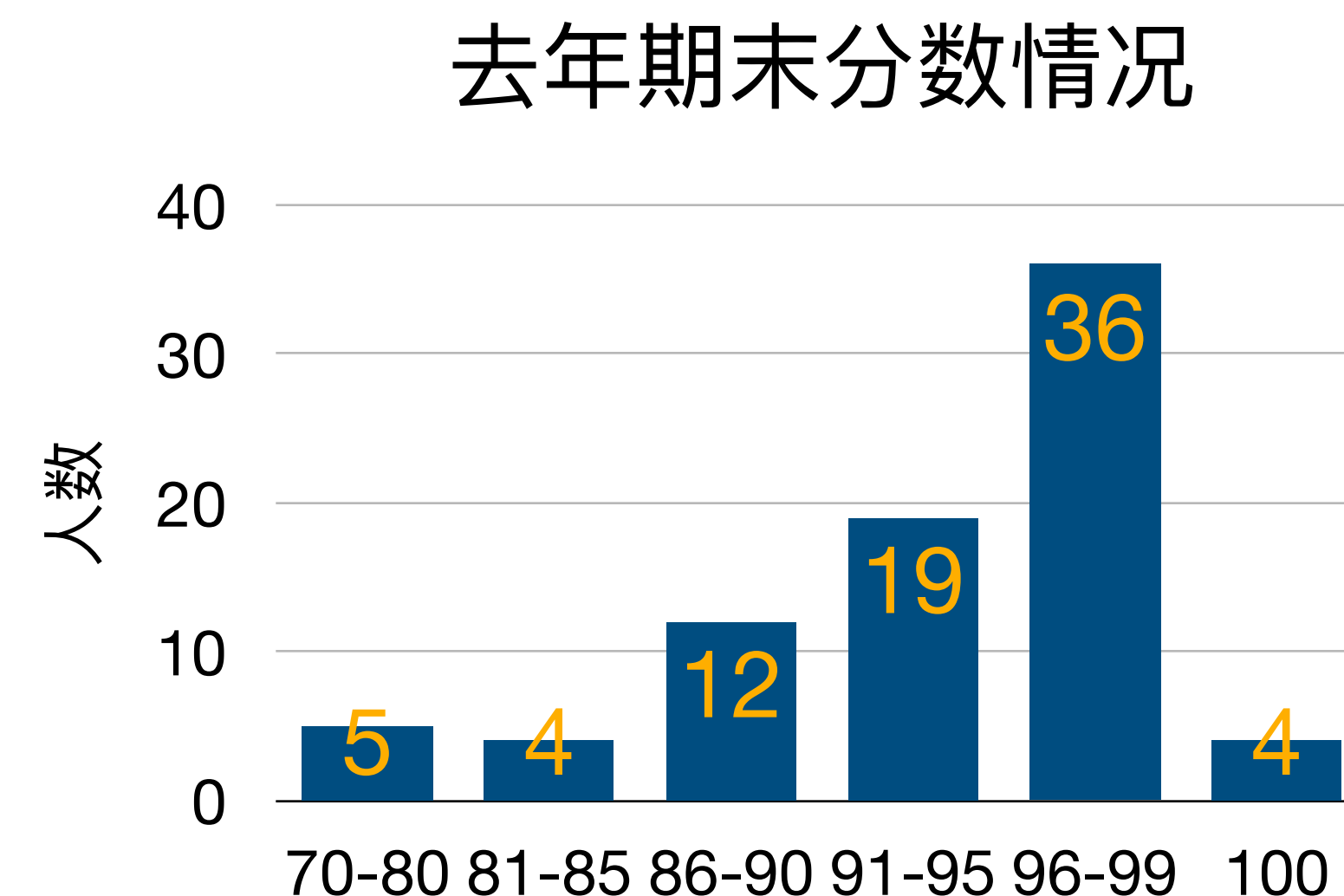
- 共7题，按计划总分50（平时分是另50）
 - 3道简单题（相对去年来说，不再有作业原题）
 - 都是确定性、非近似的经典设定下的编程问题
 - 2道中等难度题
 - 2道较难题

关于期末评分

- 考前暂不规定特别详细的评分细则，但有一些预期：

最终按总通过题数计算分数

 - 若平时满分，则：简单题 = 80，+1中等 = 85，+2 中等 = 90，+难题95/100
 - 整个评分非线性，过题比较少的同学也不会有特别低的分数
 - 通过但错误次数过多要减分（超20次减1分）
- 最后会根据同学们考后总体表现调整分数
 - 一般会往有利于同学的方向调



课程回顾

大主线

随机，近似，大数据

- 随机算法（各种采样），哈希方法：CountMin, MinHash, SimHash
- 距离与度量：

高亮 = 技巧性较高，期末考试经常出题

 - NNS：汉明近似近邻，LSH方法
 - 低维数据：格点离散化，四分树，WSPD
 - 高维数据：随机平移四分树、降维、利用稀疏性
- 大数据上的计算：数据流（sparse recovery/ ℓ_0 -sampling）、MPC

随机算法

随机算法

- 如何衡量一个随机算法的性能?
 - 精度 vs 失败概率, 即 $\Pr[\text{ALG is correct}] \geq 1 - \delta$
- 随机算法的一般设计方法
 - 设计一个无偏估计, 即让算法的数学期望性能达到精度要求
 - 得到常数概率的成功率
 - 多次独立试验提高成功率

实际中如何选取重复次数T?

如果有严格资源限制，比如时间1s

- 根据此限制最大限度设置T

如果无硬性资源限制，可以做动态平均，然后看变化趋势

- 动态平均：横轴是重复到第j次，纵轴是前j次目标函数的平均
- 当看到曲线逐步平稳，即可停止；或不看曲线但算法中设置一个阈值，当变化量连续3轮小于该阈值就可停止

Median Trick

- 现在有一个黑盒能以 $p > 0.5$ 概率正确回答某个Yes/No问题的答案
- 如何将该概率强化成任意的 $1 - \delta$?
- 期望看到：若重复 T 次，正确答案会占多数，即超过 pT 个
- 算法：重复 T 次，选取占多数（或者处于中位）的答案
- T 选多大才够？

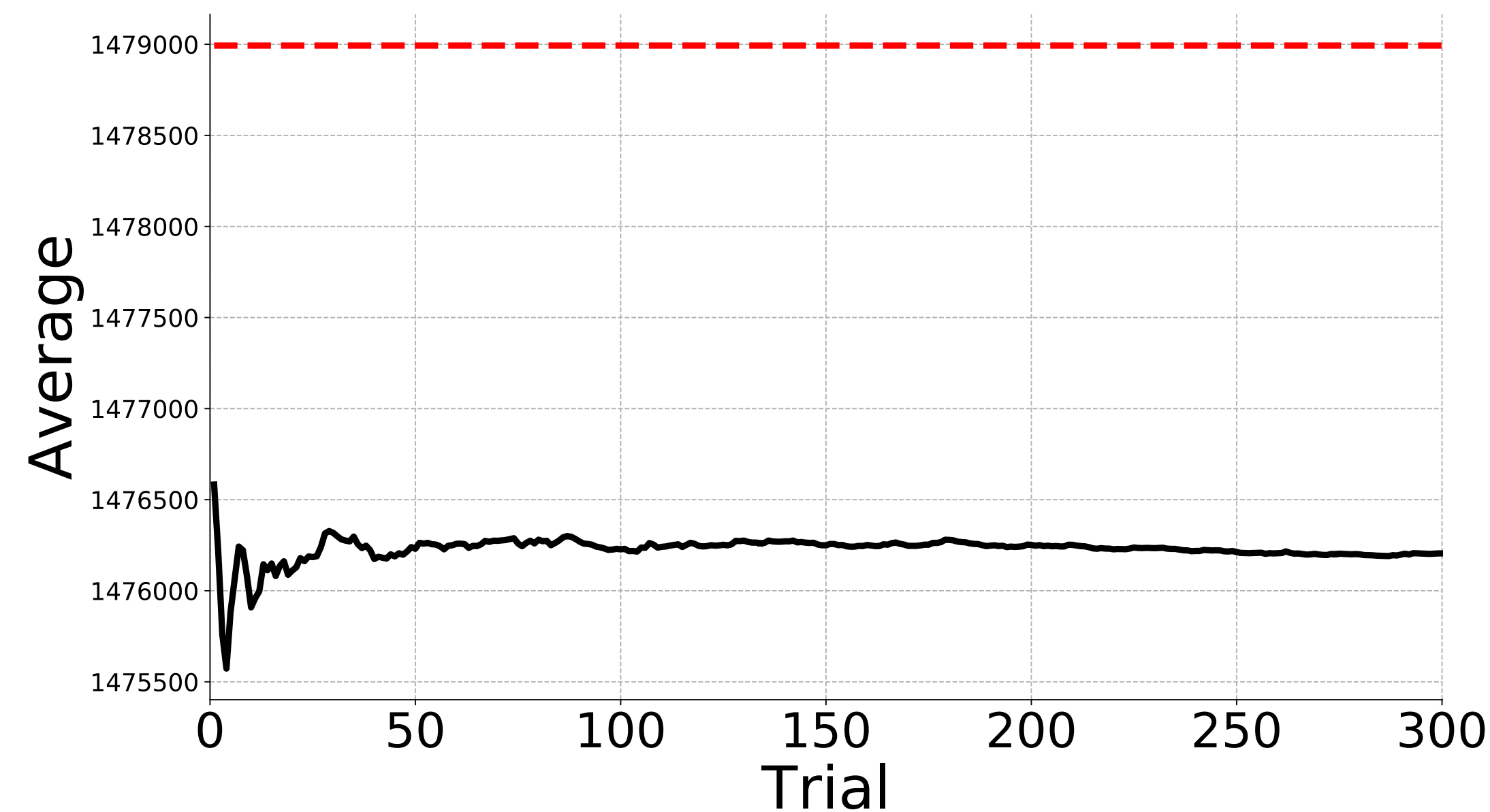
再论失败概率 δ

- 在之前的slides, 我们讨论了如何选取 δ
- 事实上: 常数 δ 就通常“不失一般性”
 - 经过 $O(\log n)$ 次重复, 可以达到 $1/\text{poly}(n)$ 的失败率
- 多大的常数?
 - 如max-cut等可以取最大来放大成功率的: 可以是任何常数
 - 如果需要用median trick, 则必须是 > 0.5 (注意严格不等号)

这对于多次运行也通常足够了, 毕竟一般只需要运行 $\text{poly}(n)$ 次

作业一：最大割随机近似算法实现、调优

- 作业一是一个实验报告（作业要求在教学网和课程网站，或点击[此处](#)）
- 给定测试数据，实现课上讲的最大割算法，并绘制重复次数-平均代价图



建议用python的matplotlib画图
作业题目里给出了画图的样例代码

作业二：最大割

- 实现最大割算法，使得在多组数据上测试都必须输出0.45近似的割
- 在openjudge评测，只有所有测试用例都通过才算通过
- 标程使用固定重复次数在设定时间内可以通过

快速测试矩阵相乘结果是否正确

- 给出三个 $n \times n$ 实矩阵 A B C ，测试是否 $AB = C$
- 确定性/暴力算法： $O(n^\omega)$ 时间，现在 $\omega \approx 2.37188$
- $O(n^2)$ 时间随机算法：如果 $AB = C$ 那么一定返回 Yes；但是 $AB \neq C$ 时可能答错
 - 随机选取一个 n 维随机 $\{0, 1\}$ 向量 x
 - 测试是否 $ABx = Cx$ ：是则输出 YES 否则 NO

达到 $\omega = 2$ 是重大 open question

可以在 n^2 时间计算 ABx ：先算 $y = Bx$ ，再算 Ay

作业三： 测试矩阵相乘是否正确

- 实现课上讲的随机检测方法
- 在openjudge评测，只有所有测试用例都通过才算通过
- 标程使用固定重复次数在设定时间内可以通过

Rejection Sampling

1/3的解法

- 考虑抛两次的情况，HH TH HT TT四种可能，以不出现TT为条件，则剩下每种出现的概率就是1/3的
- 算法：尝试连续抛掷两次硬币，若TT则重新抛，否则当HH时返回1其他返回0
- 需要抛掷多少次？（需要分析数学期望；最坏情况可以抛任意多次）

TT被reject掉了

本质上：rejection sampling实现了条件概率，但如果reject太多那么会影响性能

比如只有1%的概率不reject，那就大约采样100次才能停下来

作业四： Rejection Sampling

- 题目：推广刚刚的rejection sampling，对于一般的一个目标概率 p 生成两点分布
- 本题为代码填空/交互题，需要使用我们提供的随机性（即 $\{0, 1\}$ 均匀分布）
 - 请不要自己在函数中利用其他随机数发生器

亚线性设定

- 然而，如果不允许访问所有数据，仅可以随机访问某些元素呢？
 - 例如，这些数分布式存储在非常多的机器、硬盘上，无法载入内存
 - 但可以用较小的代价去访问指定的某第 i 个元素
 - 即，算法可以使用一种查询，该查询每次提交一个 i ，并得到 a_i 的值
- 是否能用显著小于 $O(n)$ 次的访问来得到对中位数的估计呢？例如 $O(\log n)$ ？

一个基于采样的亚线性算法

只需要 $O(1/\epsilon^2)$ 次采样就能得到 $\pm \epsilon n$ 位误差的估计

- 给定一个很小的误差限 ϵ （例如0.01）
- 算法：均匀独立采样 $T := O(1/\epsilon^2)$ 次，找这个采样上的中位数并返回
- Claim：以大常数概率，算法返回的数排在 $(0.5 \pm 2\epsilon)n$ 位上

注意到 $0.5n$ 位是“精确解”，这里有一个 $2\epsilon n$ 位次的误差

作业五： 亚线性时间估算分位点

- 作业题推广到一半的 p 分位点（中位数是 $p = 0.5$ 的特例）
- 本题为代码填空/交互题，需要用我们提供的查询器来访问数据
- 最后的性能指标不（只）看运行时间，主要看查询次数
- 在标程查询次数的一定范围内都可以通过

非均匀采样构造数据摘要问题

从均匀采样的局限性说起

- 考虑随机采样的时候，**均匀采样**是第一个该考虑的
 - 算法设计的一般原则：有简单的就不用复杂的；简单的不够再考虑复杂的
- 均匀采样的好处：一般确实可以得到**无偏估计**，即 期望 $E[X]$ 是正确的
- 不足：未必可以做到 **常数概率** 落在期望附近！

奥卡姆剃刀

总结：完整算法、结论和参数选取

- 算法：

问题：这个算法得到的 S 的大小是多少？

 - 先求使 $\text{cost}(A, c)$ 最小化的 c （可以找中位数）
 - 定义环 P_i 并在每个 P_i 上依照之前进行 m 次均匀采样得到 S_i ，并赋予合适权重
 - 将所有 S_i 求并集得到 S 返回
- 保证：对任何 q ，以概率 $1 - \delta$ 有

可以证明一共有 $O(\log n)$ 个环
要对所有环用union bound保证同时成功，需要取
 $m = O(1/\epsilon^2 \log(\log n/\delta))$

$$\text{cost}(S, q) \in (1 \pm \epsilon) \cdot \text{cost}(A, q)$$

作业六： d维1-median查询

- 作业为维针对 $d = 3$ 维的输入的近似查询

哈希方法

哈希方法

- Hash = 均匀映射，“最均匀”的是随机哈希
- 直接应用：随机负载均衡，互联网动态缓存算法consistent Hashing

Count-min Sketch

- Count-min sketch是一个数据结构，支持对于任何误差参数 $0 < \epsilon < 1$:
 - 插入 N 个 $[n]$ 上的元素后，给定一个 $x \in [n]$ ，估计 x 共出现了多少次
 - 具体来说：以大概率满足 $C \leq \hat{C} \leq C + \epsilon \cdot N$
 - 大 = $1 - 1/\text{poly}(n)$
 - “point query”
 - $\hat{C}$ 是估计量， C_x 是 x 插入次数
- 总共使用空间 $\frac{\text{poly log } n}{\epsilon}$ ，且单次查询和插入单个元素时间 $\frac{\text{poly log } n}{\epsilon}$

Count-mim Sketch: 完整算法

- 初始化:
 - 设置 $T = O(\log n)$ 个独立的随机哈希 $h^{(i)} : [n] \rightarrow [m]$, 这里 $m = 2/\epsilon$
 - 初始化 T 个 m 元 counter $C^{(i)}[1 \dots m] = 0 \quad (1 \leq i \leq T)$
- 插/删 $x \in [n]$ 时:

count-min也支持删除

 - 对每个 $i = 1, \dots, T$, 将 $C^{(i)}[h^{(i)}(x)]$ 增/减 1
- 查询 $x \in [n]$ 时: 返回 $\min_{1 \leq i \leq T} C^{(i)}[h^{(i)}(x)]$

应用：Count-min做数据流Heavy Hitter

至多有k个

- k-Heavy hitter (k-HH): A是长度N元素在[n]上的数组，求出现次数 $\geq N/k$ 的元素
- 应用
 - A是亚马逊/京东物品成交/访问记录，要快速得到当日最流行爆款商品
 - A是路由器上的IP访问日志，快速找到异常流量防DoS攻击
- 这些应用都是大数据/小内存，因此我们考虑数据流算法
- 数据流：数组A中元素以任意顺序以数据流方式给出，算法在流结束时给出k-HH

相关问题：返回top-k出现次数元素
k-HH一定在top-k，但是反之不然！

一般关注空间复杂度；
处理全流总时间一般追求 $npoly \log n$

数据流可以理解成无随机访问，类似于磁带

数据流近似k-HH：完整算法

- 输入参数：误差 $0 < \epsilon < 1$ ，整数 $k \geq 1$ ，输入元素取值范围 n
- 初始化： $M = 0$ ，一个在 $[n]$ 上的 count-min 数据结构 $P(\epsilon)$ ，一个集合 L
- 在数据流中，当 $x \in [n]$ 被插入时：
 - $M++$ ；将 x 插入 P ；若 $P.\text{count}(x) \geq M/k$ ，将 x 插入 L
 - 检查 L 中所有元素 y ，如果 $P.\text{count}(y) < M/k$ ，则把 y 从 L 删除
- 数据流结束时，返回 L

要求元素去重

重复元素会被去除

任何时候 L 中元素 $\text{count} \geq M/k - \epsilon M$ ，因此设

$$\epsilon \leq \frac{1}{2k}, \text{ 则有 } |L| = O(k)$$

作业七： k-HH的数据流算法

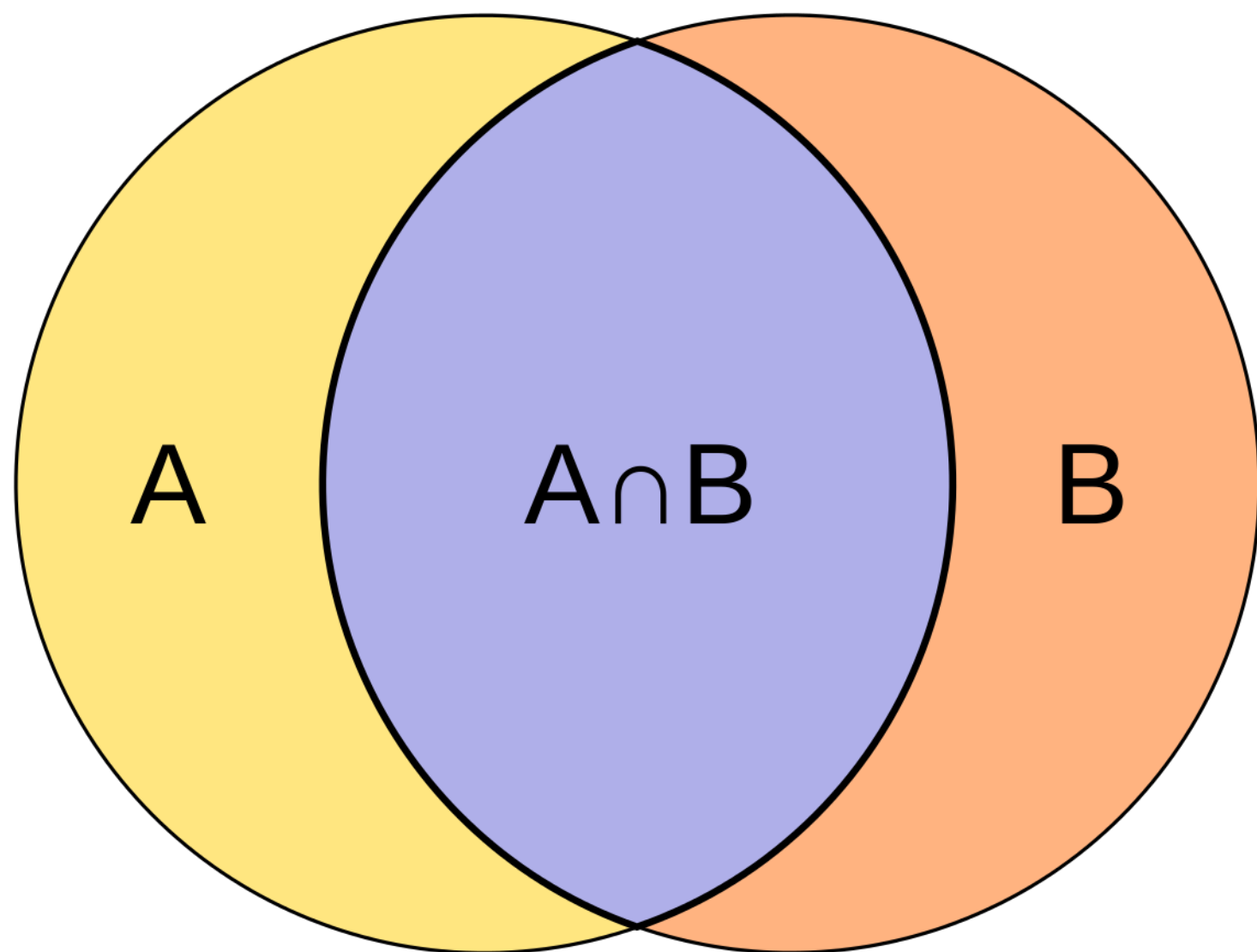
- 用刚刚介绍的基于count-min sketch的方法解决数据流近似k-HH问题
- 为了采用数据流输入和控制空间，此题需要使用提供的交互库
 - 数据流算法一般分为三个过程：初始化、更新和查询
 - 数据流开始之前运行初始化；数据流插入元素运行更新；数据流结束运行查询

Jaccard Similarity

- 两个集合 $A, B \subseteq [n]$

是一个 $[0,1]$ 的数

$$J(A, B) := \begin{cases} \frac{|A \cap B|}{|A \cup B|} & A \cup B \neq \emptyset \\ 0 & \text{otherwise} \end{cases}$$



MinHash

- 设所有的集合 A_i 的元素都来自某个universe $[n]$
- 设 $h : [n] \rightarrow [0,1]$ 为一个随机哈希 注意此处映射到实数
- 对每个 A_i , 计算 $h_{\min}(A_i) := \min_{x \in A_i} h(x)$

$$\text{Claim: } \Pr[h_{\min}(A) = h_{\min}(B)] = J(A, B)$$

转化成随机算法

- 设 $X := \begin{cases} 1 & h_{\min}(A_i) = h_{\min}(A_j) \\ 0 & \text{oth.} \end{cases}$, 则 $\Pr[X = 1] = J(A_i, A_j)$

- 完整算法（多次试验取平均值）：

如何实现？

- 采用 T 个独立的随机哈希 $h^{(1)}, \dots, h^{(T)}$

- 对每个 A_i 和 $h^{(t)}$, 计算 $h_{\min}^{(t)}(A_i)$

$O(|A_i| \cdot T)$ 时间

计算的是 $h_{\min}^{(t)}$ 相等的 t 所占比例

- 对查询 A_i, A_j , 计算 $\frac{1}{T} \cdot |\{t \in [T] : h_{\min}^{(t)}(A_i) = h_{\min}^{(t)}(A_j)\}|$

$O(|T|)$ 时间

作业八：用MinHash近似Jaccard Similarity

Cosine Similarity

- 考虑两个向量 $x, y \in \mathbb{R}^d$, 定义 $\sigma(x, y) := \frac{\langle x, y \rangle}{\|x\|_2 \|y\|_2}$
 $\sigma(x, y) = \cos(\theta(x, y))$
- 典型应用：文本相似度（TF-IDF）
 - TF = term frequency：一个单词在文档中出现的频率
 - IDF = inverse document freq.： $\log(\text{总文档数} / \text{单词出现在多少文档})$
 - 对单词 w , $\text{TF-IDF}(w) = \text{TF}(w) * \text{IDF}(w)$, 做成一个下标是单词、值TF-IDF向量

一般非常的高维、稀疏

Cosine Similarity的哈希： SimHash

- 类似于MinHash, 我们想找到一个 h , 使得 $\Pr[h(x) = h(y)]$ 可以反映 $\sigma(x, y)$
- 算法：

如何生成：生成 d 个独立的标准正态变量，然后将该随机向量归一化
- 生成一个 d 维随机高斯向量 w (等价于在 d 维单位球面取一个均匀随机点)
- $h(x) := \text{sgn}(\langle w, x \rangle)$, 其中 $\text{sgn}(x) = \begin{cases} 1 & x \geq 0 \\ 0 & \text{otherwise} \end{cases}$ 是符号函数
- Claim: $\forall x, y \in \mathbb{R}^d, \Pr[h(x) \neq h(y)] = \frac{\theta(x, y)}{\pi}$

$\theta(x, y) \in [0, \pi]$ 是 x 和 y 的向量夹角

近似近邻搜索

Hamming空间找近似最近邻：专门算法

- 设有 n 个 d 维的Hamming空间 $\mathbb{H}^d$ 上的点
- 性能要求：
 - 误差参数 $\epsilon > 0$
 - 预处理时间 $O(dn + n^{1+1/(1+\epsilon)})$
 - 给定任何 $q \in \mathbb{H}^d$ ，可在 $O(n^{1/(1+\epsilon)})$ 时间找到 $(1 + \epsilon)$ -近似最近邻

这里典型情况要求 $\epsilon \geq 1$

即：若最近邻距离是 r ，则返回的点距离 $\leq (1 + \epsilon)r$

作业九：Hamming空间近似最近邻查询

一般LSH的定义及基于LSH的近似近邻查询算法

- (c, r, p_1, p_2) -LSH是一族定义在 $\mathbb{R}^d$ 上的随机哈希函数 h , 满足:
 - 对满足 $\|x - y\| \leq r$ 的 x, y , 有 $\Pr[h(x) = h(y)] \geq p_1$ 距离近的点有大概率在同一个bucket
 - 对满足 $\|x - y\| > cr$ 的 x, y , 有 $\Pr[h(x) = h(y)] \leq p_2$ 距离远的点有小概率在同一个bucket

Claim: 定义 $\rho := \frac{\log(p_1)}{\log(p_2)}$, 有 $n^{1+\rho}$ 预处理、 n^ρ 查询时间做 c -近似判定的算法

ρ 成为了关键参数

LSH举例：Hamming空间

Claim: 定义 $\rho := \frac{\log(p_1)}{\log(p_2)}$, 有 $n^{1+\rho}$ 预处理、 n^ρ 查询时间做 c -近似判定的算法

- Hamming空间的LSH

- 构造：从 $i \in [d]$ 中均匀随机选取一个 $\hat{i}$, 然后定义 $h : x \mapsto x_{\hat{i}}$

- $\Pr[h(x) = h(y)] = 1 - \text{dist}_H(x, y)/d$, 所以 $p_1 = 1 - r/d$, $p_2 = 1 - cr/d$

- $\rho = \frac{\log(1 - r/d)}{\log(1 - cr/d)} \leq O(1/c)$

(c, r, p_1, p_2) -LSH 是一族定义在 $\mathbb{R}^d$ 上的随机哈希函数 h , 满足:

- 对满足 $\|x - y\| \leq r$ 的 x, y , 有 $\Pr[h(x) = h(y)] \geq p_1$
- 对满足 $\|x - y\| > cr$ 的 x, y , 有 $\Pr[h(x) = h(y)] \leq p_2$

LSH举例：单位球上向量的欧氏距离的LSH

- 设数据都是单位球 $\mathcal{S}^{d-1}$ 上的向量，用 ℓ_2 也就是欧氏距离度量距离
- 构造：直接利用cosine similarity的SimHash
- $\Pr[h(x) = h(y)] = 1 - \theta(x, y)/\pi$
 - 所以 $p_1 = 1 - 2 \arcsin(r/2)/\pi$, $p_2 = 1 - 2 \arcsin(cr/2)/\pi$
 - $\rho = \frac{\log(1 - 2 \arcsin(r/2)/\pi)}{\log(1 - 2 \arcsin(cr/2)/\pi)} \leq O(1/c)$

LSH举例： Cross-polytope LSH

[Practical and Optimal LSH for Angular Distance, Andoni et al.]

- 对单位球 $\mathcal{S}^{d-1}$ 上欧氏距离的情况，有一种SimHash的推广
- 设 $M \in \mathbb{R}^{d \times d}$ 为一个随机高斯矩阵，即每个元素是独立的 $\mathcal{N}(0,1)$
- 对 $x \in \mathcal{S}^{d-1}$ ，哈希函数计算 $y := Mx/\|Mx\|_2$ ，然后找离 y 最近的 $\{\pm e_i\}_{i \in [d]}$ 返回

$\{e_i\}$ 是标准单位基

- 当 M 的行数 = 1时退化成SimHash
- 可以得到 $\rho = 1/c^2$ ，比SimHash的 $1/c$ 要好

作业十： 欧氏单位球上的近似近邻查询

- 可自由尝试：
 - 标程用cross-polytope LSH, 用Hadamard优化
 - simhash也可以过
- 可选探究：一般LSH的方法 vs 前面介绍的专用算法来做作业九

低维欧氏空间上的近似算法技术

线性时间 $(1 + \epsilon)$ -近似直径

T 可通过选取任意点 u , 求 u 到最远点的距离得到 (见第一讲)

即满足 $1/2 \cdot \text{diam}(P) \leq T \leq \text{diam}(P)$

- 先 $O(n)$ 时间找一个直径的2-近似值 T
- 作 $\ell := \epsilon \cdot T / \sqrt{2}$ 的网格, 并round到中心
- 因此每个点移动了 $\leq \sqrt{2}\ell \leq \epsilon \cdot \text{diam}(P)$
- 因此新点集 P' 满足

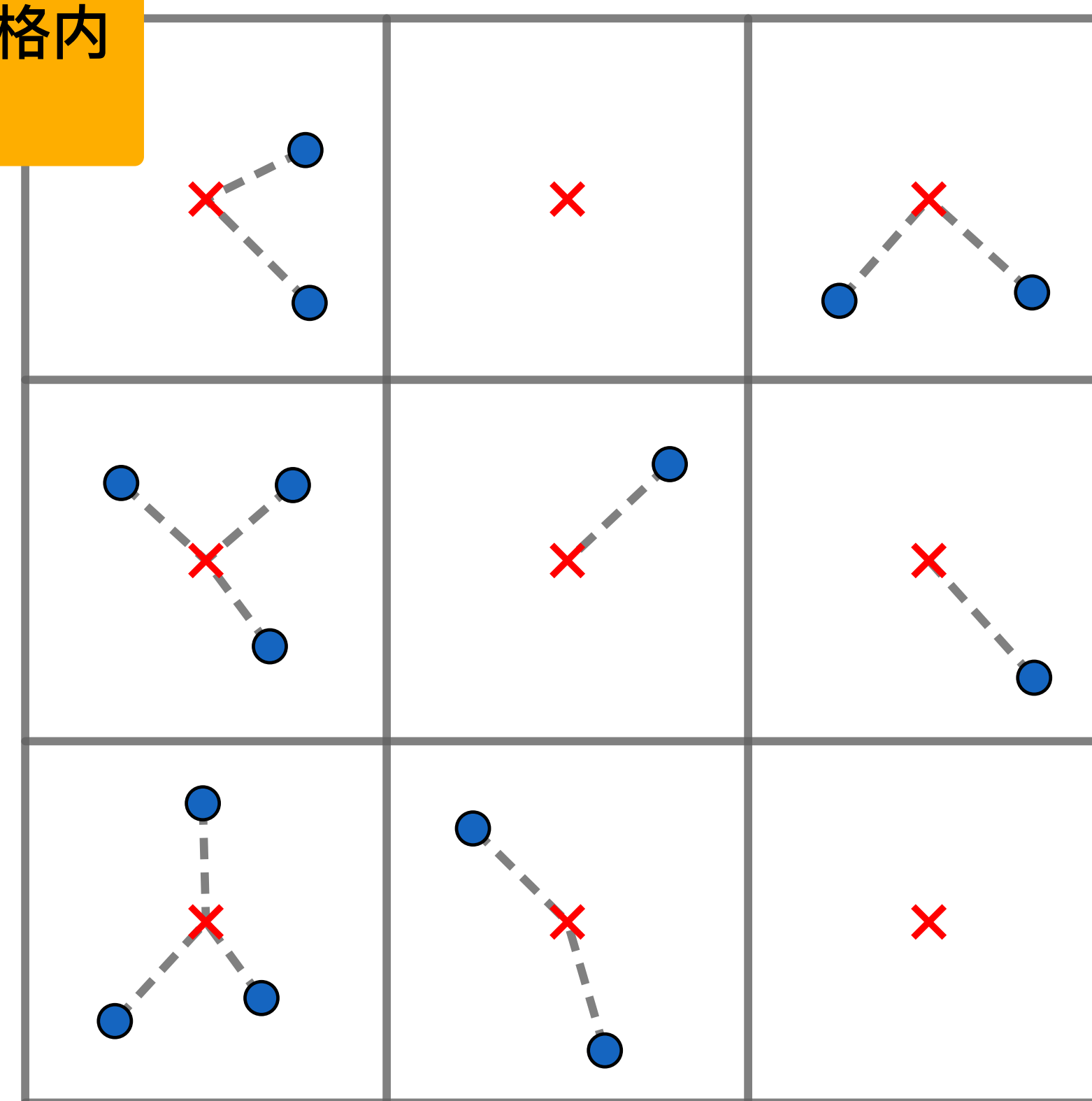
可在 $O(nd \log n)$ 时间构造 P'

$$\text{diam}(P') \in (1 \pm \epsilon) \cdot \text{diam}(P)$$

- 算法: 在 $|P'|$ 上暴力求直径, 复杂度 $|P'|^2 \cdot d$

P' 所有点都在 $\text{diam}(P') \times \text{diam}(P')$ 大方格内, 小方格 $\ell \geq \Omega(\epsilon \cdot \text{diam}(P))$, 故 $|P'| \leq (O(1/\epsilon))^2$

事实上可以是方格内任意确定点



总复杂度: $O(nd \log n) + (d/\epsilon)^{O(d)}$

作业十一：近似求欧氏点集直径

Idea 2: 递归格点离散化, 四分树划分

以二维为例

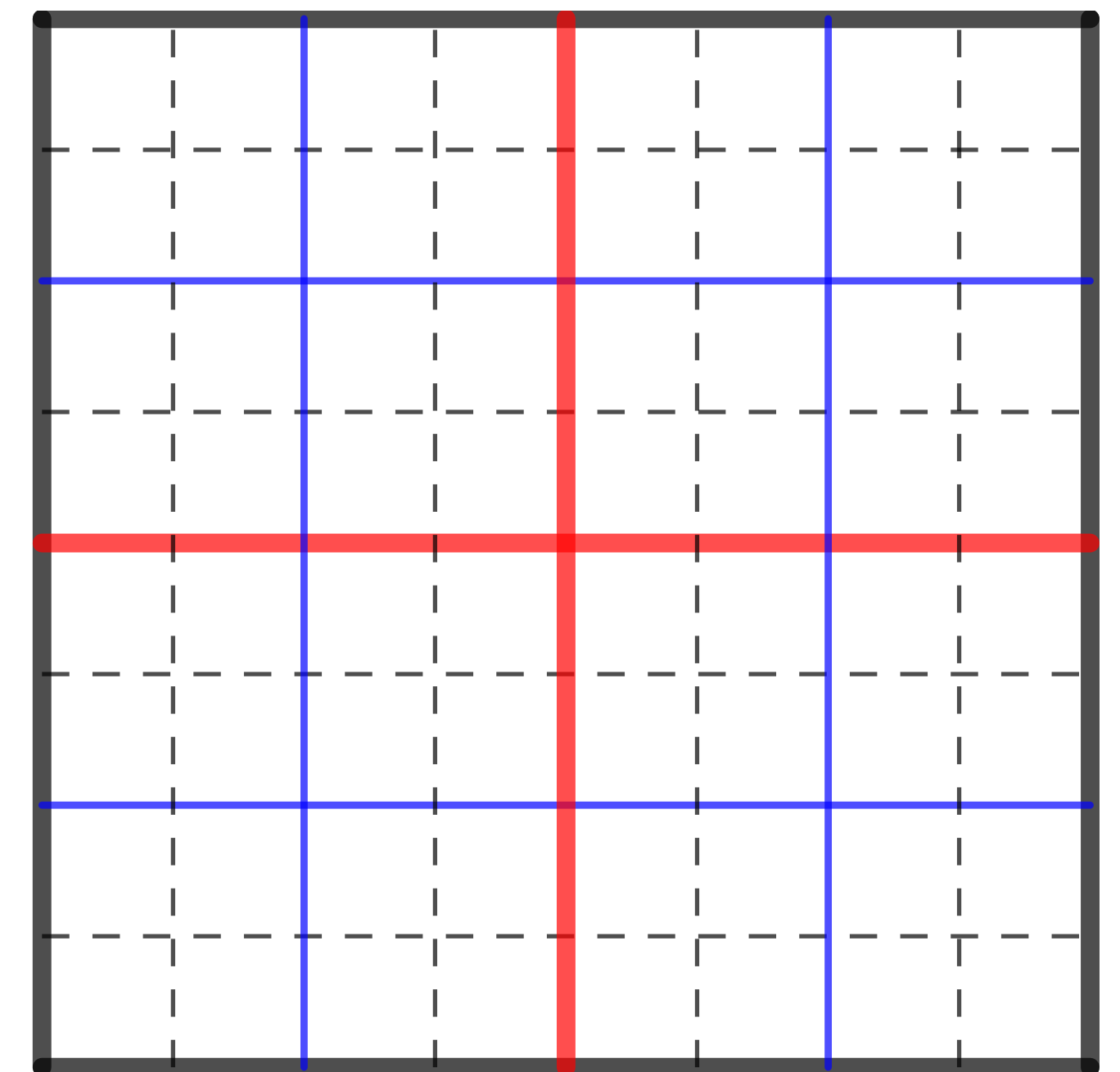
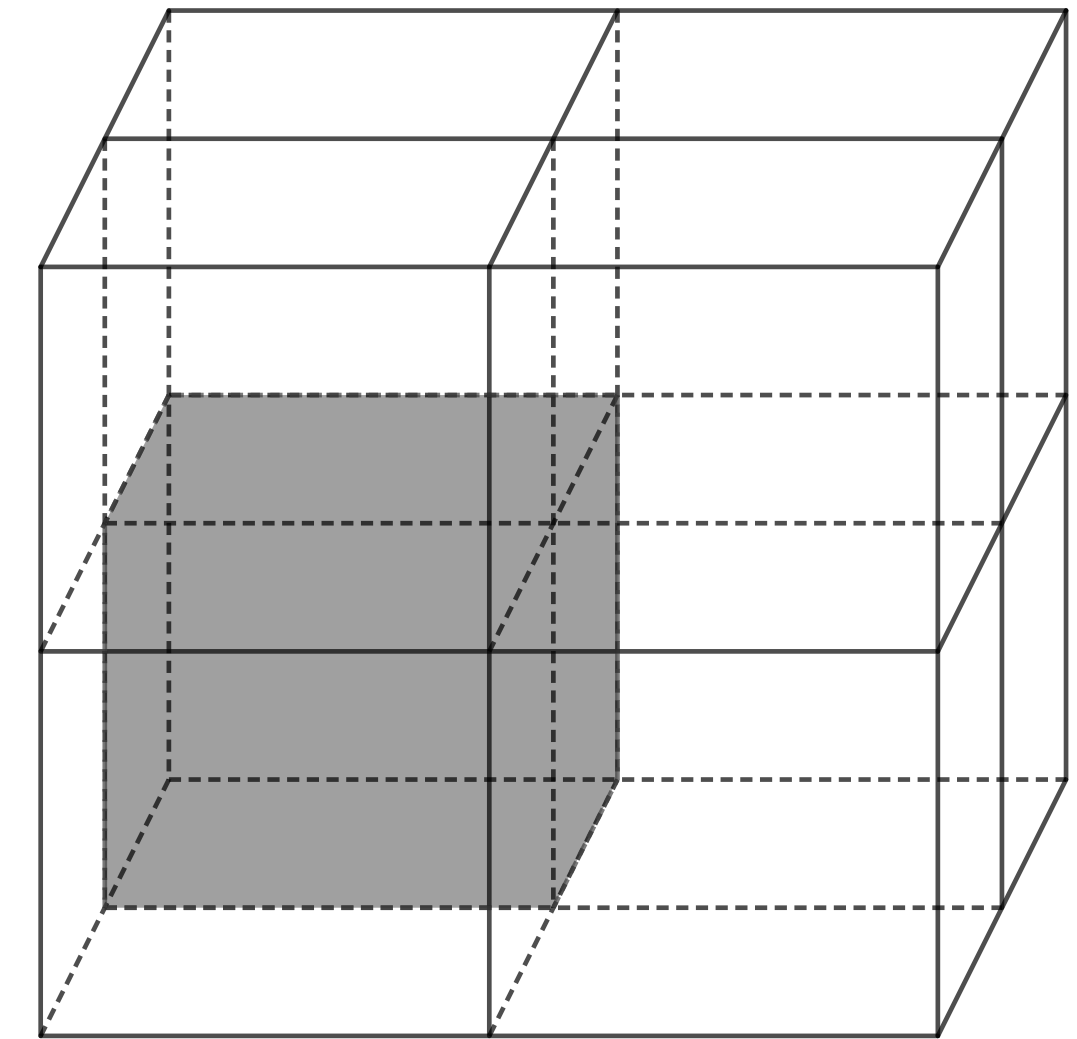
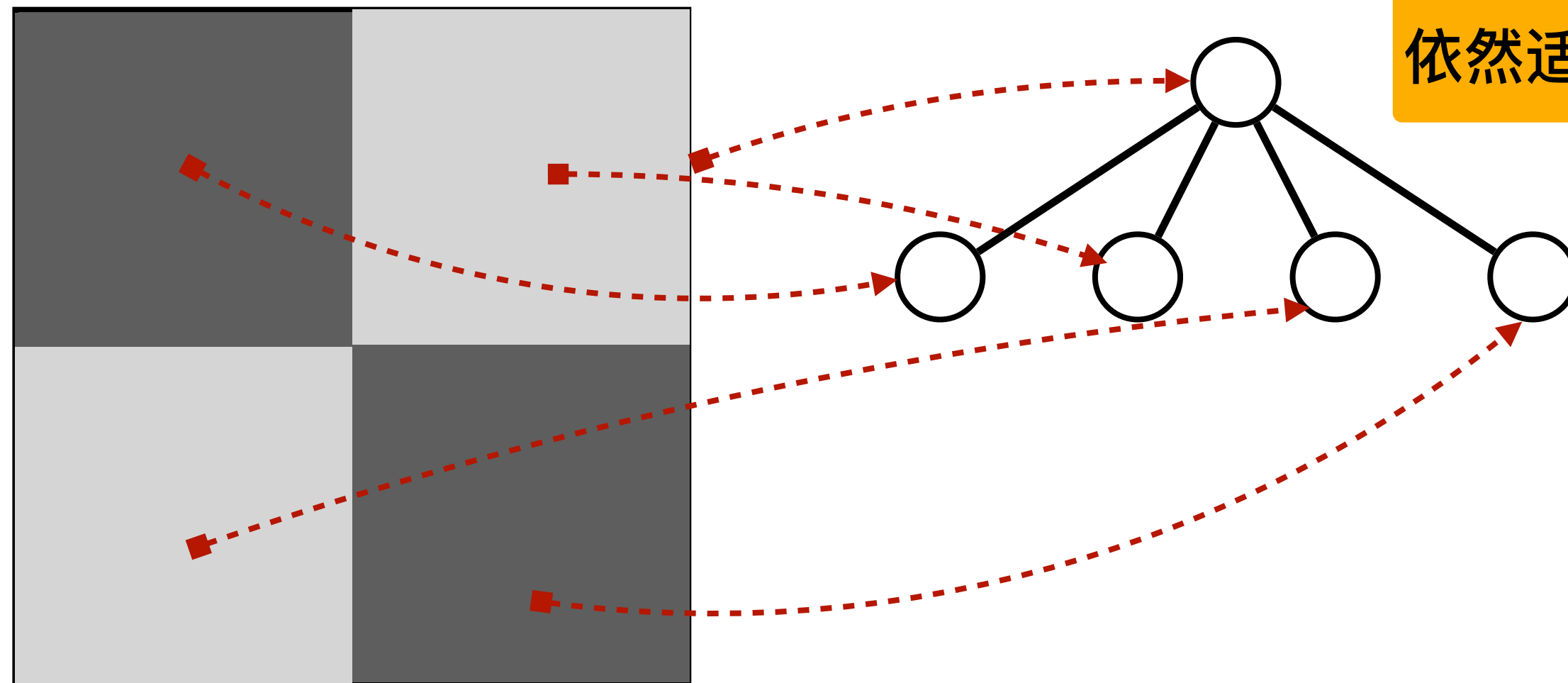
d维叫“d维四分树”

d维是从 $[\Delta]^d$ 划分

- 从最大的 $\Delta \times \Delta$ 的正方形开始
- 递归将当前 $\square$ 等分成4个边长为一半的 $\square$
- 组织成树形结构, 直到只含单点停止, 至多 $\log_2 \Delta$ 层

d维分成 2^d 个边长为一半的 $\square$

依然适用于d维



利用四分树进行 ϵ -近似最近邻查询

- 问题:

- 对输入点集 $P \subseteq [\Delta]^d$ 进行预处理

即 $\hat{q}$ 到 q 的距离是 q 到 q 在 P 最近邻距离的 $(1 + \epsilon)$ 倍

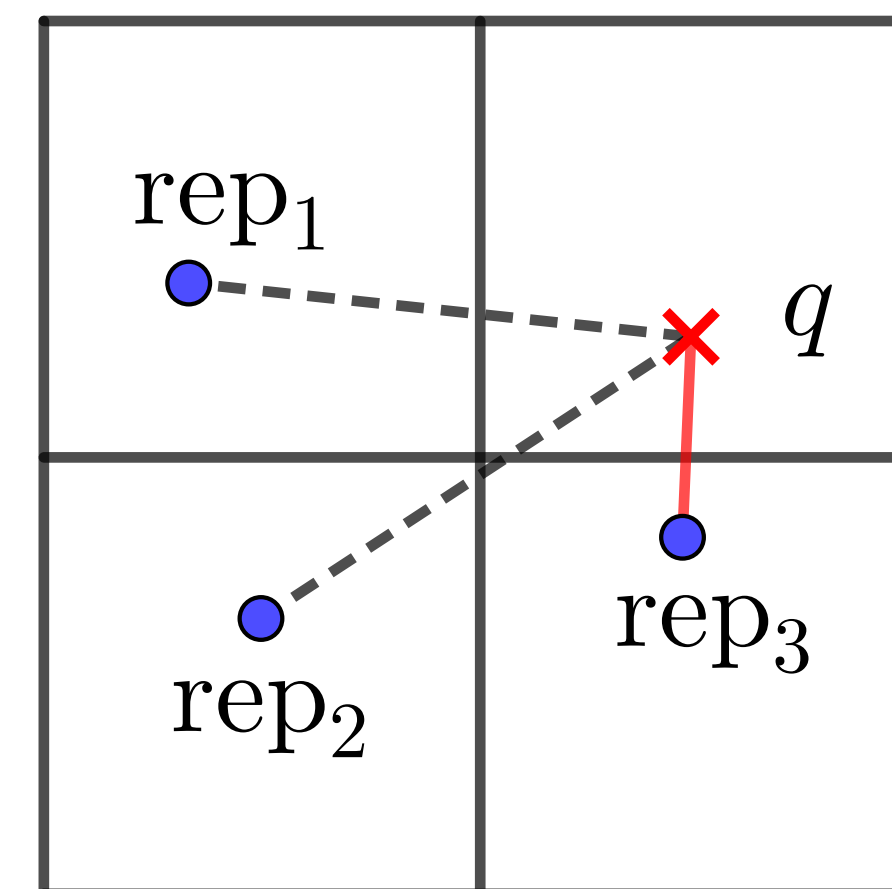
- 对任意 $q \in [\Delta]^d$ 回答 $\hat{q} \in P$, 使得 $\|\hat{q} - q\| \leq (1 + \epsilon) \cdot \min_{q' \in P} \|q' - q\|$

- 预处理: BuildTree($\square$, P), 这里调用 $\square = [\Delta]^d$

- 我们将给出一个 $O(\epsilon^{-d} \log \Delta)$ 时间的查询算法返回满足要求的 $\hat{q}$

完整算法

输入：查询点 q ，误差参数 $0 < \epsilon < 1$ ；查询所需时间 $O(\epsilon^{-d} \log \Delta)$



设集合 A_i 为需要检查的四分树第 i 层的格子的集合 ($0 \leq i \leq \log \Delta$)

初始 A_0 放入最大格子 $\square = [\Delta]^d$, $A_i = \emptyset$ ($1 \leq i \leq \log \Delta$), $r = \infty$, $\hat{q} = \text{NULL}$

for $i = 1, \dots, \log \Delta$

i 是层数，按层进行迭代

$\hat{q}$ 是当前解， r 是 $\|q - \hat{q}\|$

设 S 为所有 A_{i-1} 的格子的 2^d 个子 $\square$ 的集合

更新当前解为本层最近的rep

对每个 $\square_w \in S$ ，若 $\|q - \text{rep}_w\| < r$ ，更新 $r = \|q - \text{rep}_w\|$ 及 $\hat{q} = \text{rep}_w$

对每个 $\square_w \in S$ ，若 $r > (1 + \epsilon) \cdot (\|q - \text{rep}_w\| - \text{diam}(\square_w))$ ，将 $\square_w$ 放入 A_i

此处是“需要放入 $\square_w$ ”的判据，因此是 $>$

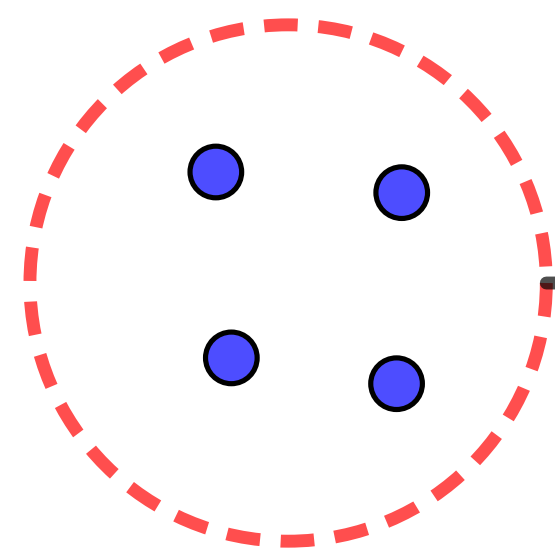
返回 $\hat{q}$

作业十二：欧氏空间近似最近邻查询

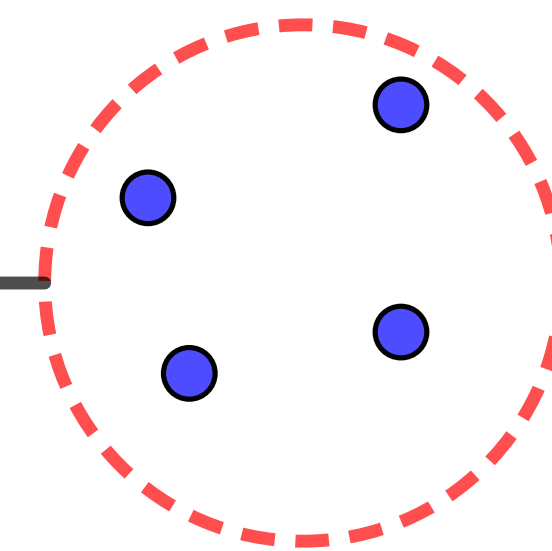
Idea 3: 考虑点对的离散化, WSPD

直观讨论

- 输入点集 $P \subseteq [\Delta]^d$, 设 $|P| = n$
- 考虑所有点对的距离 (共有 $O(n^2)$ 对), 试图将点对距离离散化
- 直觉: 设有 $S, T \subseteq P$ 两组点, 且他们距离很远
 - 那么可以把 S 和 T 分别“缩点”, 从而将 $S \times T$ 上的距离等于同一个值



即使不显式缩点, 左右两边点的距离相对来说都差不多; 例如左右两圆直径=1, 但相距100



WSPD

定义

- 一个 ϵ -WSPD是一系列 $\{\{A_i, B_i\}\}_i$, 使得

点集之间的距离等于最近点的距离, 即

$$\text{dist}(S, T) := \min_{x \in S, y \in T} \|x - y\|$$

- $\forall i, A_i, B_i \subseteq P$ 且 $\max(\text{diam}(A_i), \text{diam}(B_i)) \leq \epsilon \cdot \text{dist}(A_i, B_i)$ 即两组点 A_i, B_i 距离很远

- $\bigcup_i A_i \times B_i = P \times P$, 即任何 $p \neq q \in P$ 可以找到某个 i 使得 $p \in A_i, q \in B_i$

可以理解为对 $O(n^2)$ 个距离对的划分



基于四分树的WSPD：完整算法

WSPD用四分树的 $\square$ 表示，总复杂度 $O(n\epsilon^{-d} \log \Delta)$ ，所得WSPD有 $O(n\epsilon^{-d} \log \Delta)$ 项

全局对 $\Delta = [\Delta]^d$ 调用WSPD($\square, \square$)

WSPD($\square_u, \square_v$)

定义： $\delta(\square_u) = \text{diam}(\square_u)$ 若 $|\square_u \cap P| \geq 2$ ，否则 $\delta(\square_u) = 0$

if ($u = v$ and $\delta(\square_u) = 0$) return // 单点 $\square$ 不要配对到自己身上

if ($\delta(\square_u) < \delta(\square_v)$) swap u, v

if ($\delta(\square_u) \leq \epsilon \cdot \text{dist}(\square_u, \square_v)$)

直接看 $\square$ 而不是 $P \cap \square$

return $\{ \{ \square_u, \square_v \} \}$

return $\bigcup_{i=1}^{2^d} \text{WSPD}(\square_{u_i}, v)$

运行至此， $\delta(\square_u) \geq \delta(\square_v)$ ，因此 $\{ \square_u, \square_v \}$ 确实满足WSPD条件，又因为算法会尝试一直递归到底，因此**确保最后输出的是合法WSPD**

$\square_{u_i}$ 是 $\square_u$ 的子 $\square$

WSPD应用：精确求最近点对

- 这是一个 $O(n \cdot 2^d \log \Delta)$ 时间的精确算法：
 - 构造 $\epsilon < 1$ 的WSPD $\{\{A_i, B_i\}\}_i$ $O(n \cdot 2^d \log \Delta)$ 时间
 - 对所有的 $\{A_i, B_i\}$ ，若 $|A_i| = |B_i| = 1$ ，记录 A_i 和 B_i 中唯一点的距离
 - 在所有距离中的最近点对就是答案 $O(n \cdot 2^d \log \Delta)$ 时间
- 原因：设最近点对是 (p, q) 且 $p \in A_i$ ， $q \in B_i$ ，则 $|A_i| = |B_i| = 1$

因此算法不会
遗漏最近点对

若有多余点 x 在 A_i ，则容易看出 $\|p - x\| < \|p - q\|$ ，矛盾



作业十三：最近点对精确求解

WSPD应用：求 $(1 + \epsilon)$ -Spanner

$$t \geq 1$$

- t -Spanner是数据集 $P \subseteq [\Delta]^d$ 的一个欧氏图的子图 H
 - 欧氏图：点集还是 P ，边集是 $P \times P$ ，边权重是端点间的欧氏距离
 - 子图：点集相同，边集是子集
 - t -Stretch：要求对任何 $x, y \in P$ 有 $\|x - y\| \leq \text{dist}_H(x, y) \leq t \cdot \|x - y\|$
 - 追求：给定 t ， H 满足 t -stretch条件，且边数尽量少
- 可看作是欧氏图的稀疏“压缩”，后续可使用稀疏图上的算法来解决欧氏问题

dist_H 代表 H 上的最短路距离

Spanner算法

- 算法：先求 $\epsilon/8$ -WSPD，再对每个 A_i, B_i 都找代表点 rep_1 和 rep_2 连边
- 因此总共是 $O(n\epsilon^{-d} \log \Delta)$ 条边

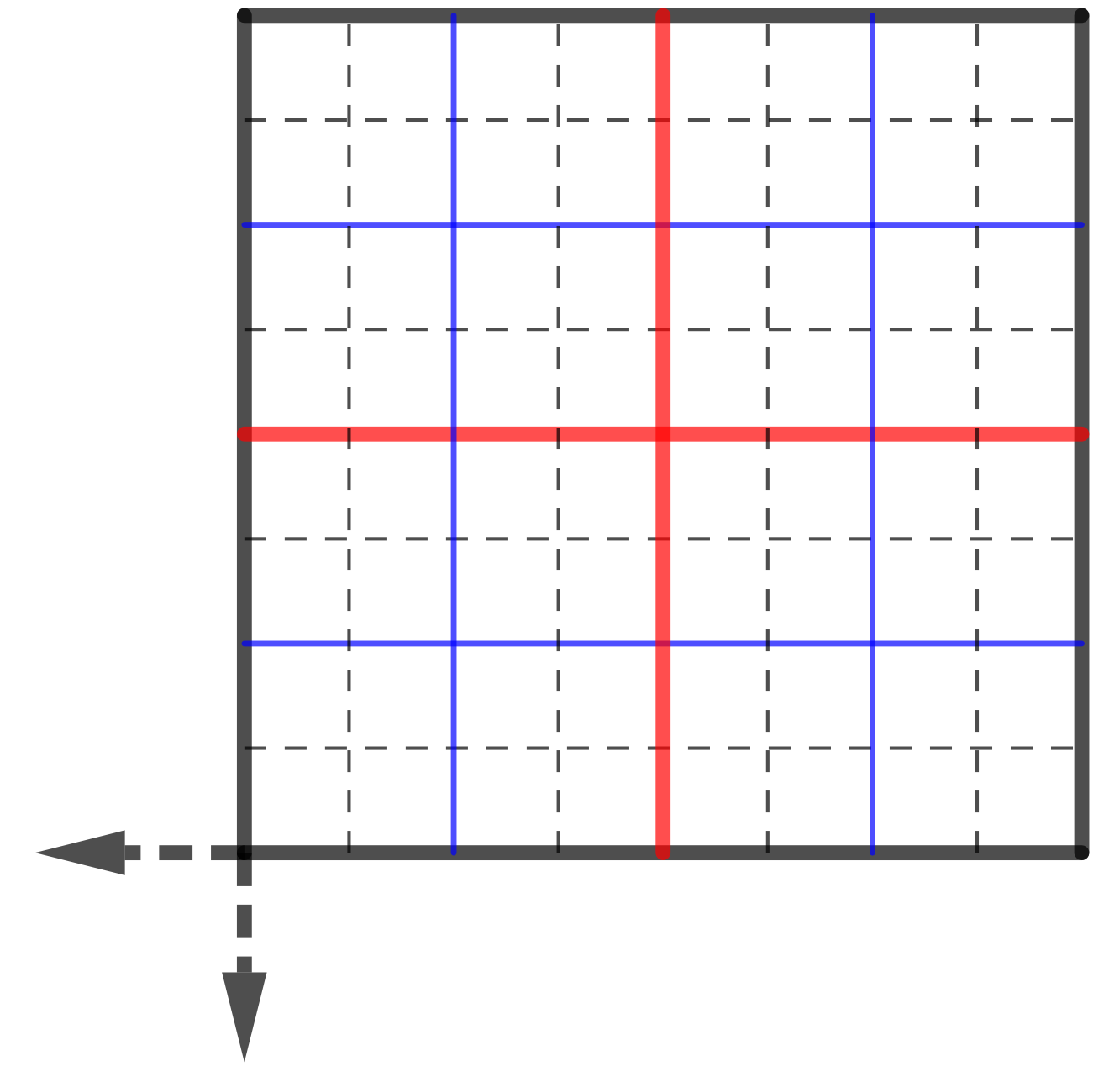
作业十四： $(1 + \epsilon)$ -近似欧氏最小生成树

高维欧氏空间上的近似算法技术

一个通用技术：随机平移四分树

每维独立生成 $[0, \Delta]$ 随机数

- 选取 $[0, \Delta]^d$ 上的均匀随机向量 v
- 构造四分树 $\text{BuildTree}([0, 2\Delta]^d, P)$ 后，将整个树平移 $-v$
 - 也可以看作 $\text{BuildTree}(\square = [-v, 2\Delta - v]^d, P)$
- 回忆：构造四分树复杂度为 $O(nd \log \Delta)$ ，不需要 2^d
- 我们将利用这同样的构造给出若干不同的应用方式



$[0, 2\Delta]$ 确保平移后依然覆盖点集

Tree Embedding

- Tree embedding: 将数据集 P 从 $\mathbb{R}^d$ 映射到一棵树 $T(V, E)$ 上, 其中 $P \subseteq V$
 - 旨在用树上的距离来“近似”/“代替”欧氏距离
 - 首先要求 $\forall x, y \in P, \text{dist}_T(x, y) \geq \|x - y\|$
- 主要性能衡量叫distortion:

是 ≥ 1 的量, 越小越好

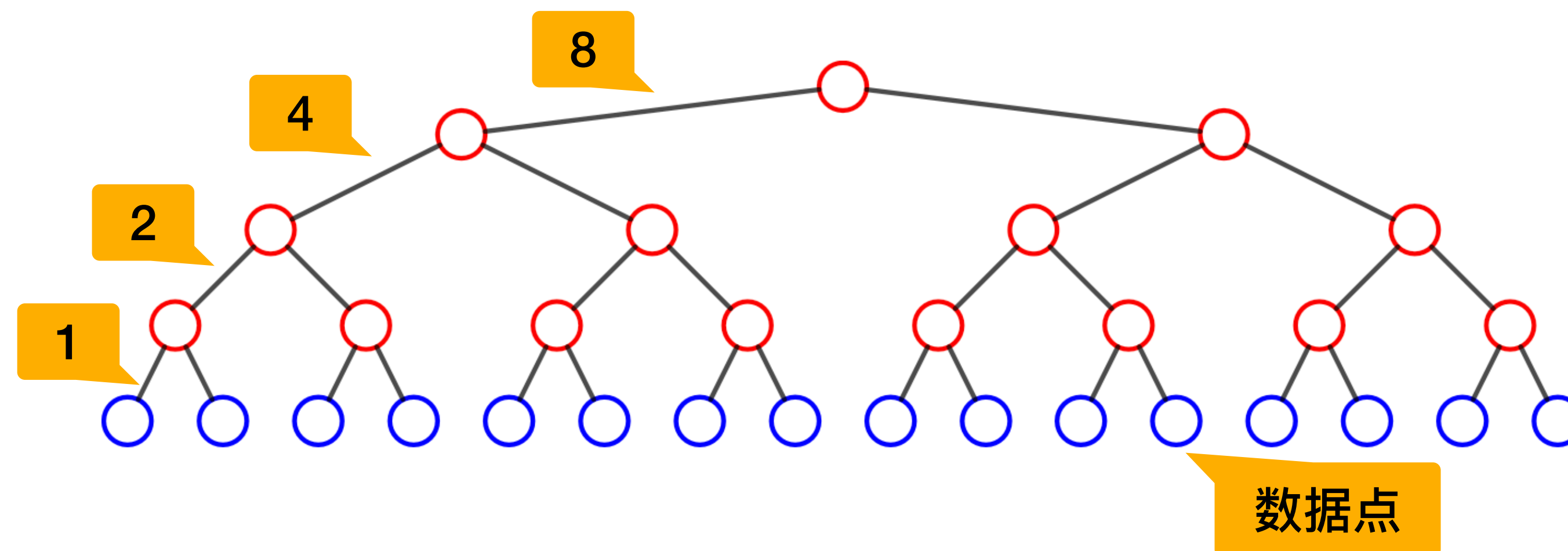
$$\max_{x \neq y \in P} \frac{\text{dist}_T(x, y)}{\|x - y\|}$$

Tree Embedding额外要求: 2-HST

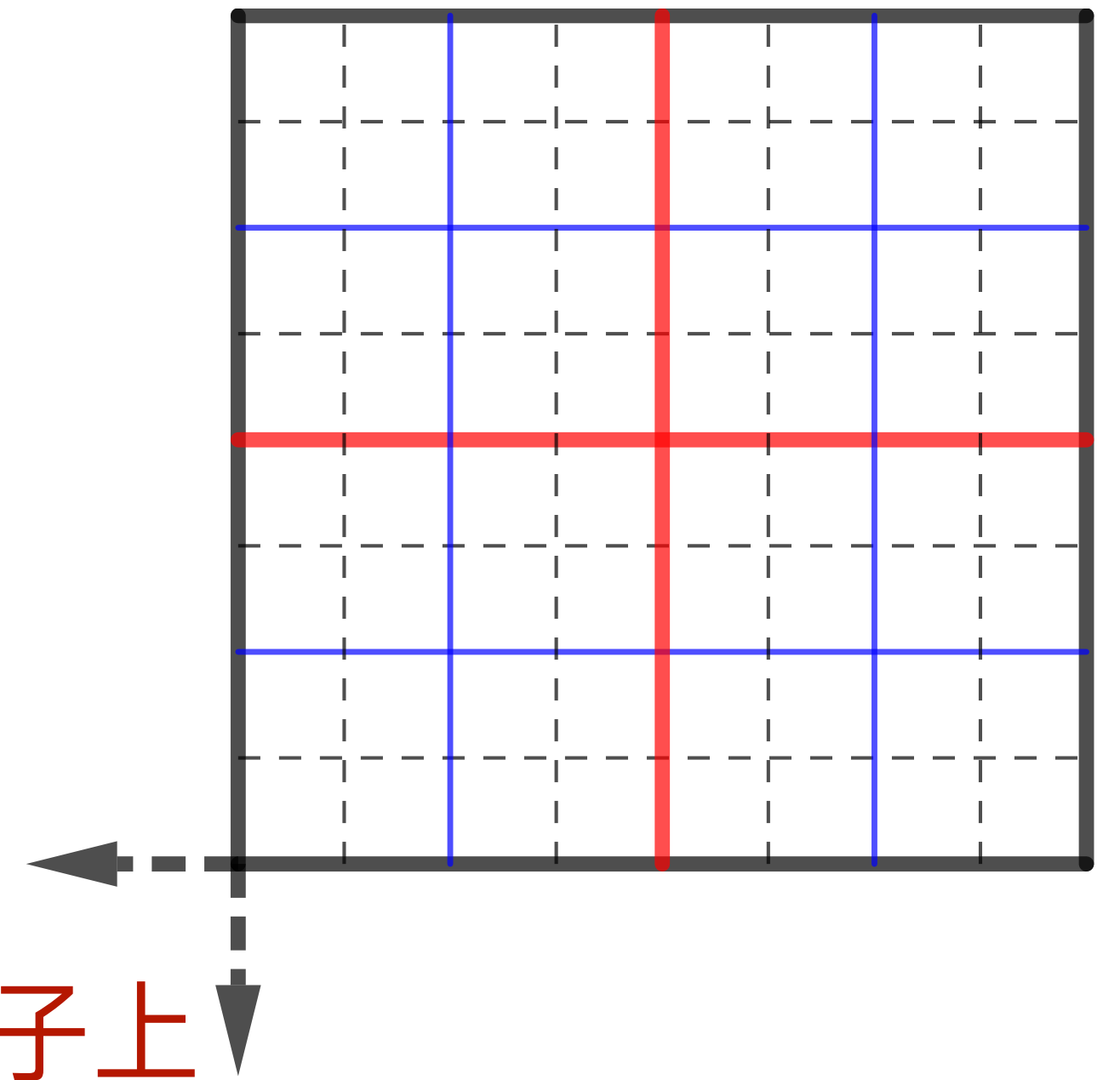
HST = hierarchically separated tree

一般要求这个树有更好的结构:

- 数据集P都在树的叶子上, 且所有叶子高度相同 (所以内部点都只是辅助点)
- 考虑任何一条从叶子到根的路径, 边权依次乘以2



构造算法和结论



- 首先构造随机平移四分树
- 定义树 T : 每个 $\square$ 定义一个树节点, 数据点都放在树的叶子上
- 定义边: 每个 $\square$ 连接到所有的直接孩子, 赋予 $\sqrt{d} \cdot 2^i$ 的权值, 设 $\square$ 边长 2^i
- 对于 $x, y \in P$ (都是树的叶子), 距离 $\text{dist}_T(x, y)$ 就是树上的距离
- 结论: $\forall x, y \in P, \text{dist}_T(x, y) \geq \|x - y\|, \mathbb{E}[\text{dist}_T(x, y)] \leq O(dh) \cdot \|x - y\|$

下界是确定性的

上界是随机的

h 是四分树的高度

作业十五：Tree Embedding近似高维欧氏最小权匹配

高维Spanner结论与算法

- 结论：存在一个 $O(d)$ -spanner，构造时间/边数 $O(n \log(n) \log(\Delta))$
- 算法：初始化spanner边集 $H \leftarrow \emptyset$
 - 设 $m \leftarrow O(\log n)$ ，独立采样 m 个随机平移四分树 $T_1, \dots, T_m$
 - 对每个 $j \in [m]$ ，每个树上的节点 $u \in T_j$
 - 设 P_u 为 u 含有的数据点集合，将 P_u 上的任意一个星形图加入 H
- 边数、时空复杂度的分析？

星形图：除了一个树根外其余全是叶子的树

作业十六： 高维Spanner近似MST

- 潜在的也可尝试tree embedding，但实际近似比可能略差（标程采用spanner）

Range Query问题

- 输入：在 $\mathbb{R}^d$ 中的 n 个数据点集合 P ，参数 $r > 0$
- 查询：给定 $q \in \mathbb{R}^d$ ，估计 $|B(q, r) \cap P|$ 的点数
- 一般而言： 查询要求估计 $f(B(q, r) \cap P)$ ，其中 f 是composable函数
 - 称一个函数 $f: 2^P \rightarrow \mathbb{R}$ 是composable的，若对任意的 S 及其划分 $S_1, \dots, S_t$
 $f(S)$ 可以从 $f(S_1), \dots, f(S_t)$ 的值推出
 - 例如sum、max、min等

近似保证的定义、结论

- 引入记号 $B_P(q, r) := B(q, r) \cap P$
- 这里考虑下面这种 α -近似: 实际返回的 E 满足 $|B_P(q, r)| \leq E \leq |B_P(q, \alpha r)|$

即类似于近似近邻查询的要求, 允许将 αr 内的点计入

- 我们介绍一种基于随机平移网格的算法

只需要网格, 不需要四分树,
但必须随机平移

- 查询达到 $O(\alpha)$ -近似、期望用时 $O(2^{d/\alpha^{2/3}} \text{poly}(d))$

主要针对大近似比

即牺牲精度改进指数上的依赖
选取足够大的 α 可以做到 $d^{1.5}$ -近似, $\text{poly}(d)$ 时间
这个 $\alpha^{2/3}$ 不一定是紧的, 只是目前的分析能达到的界

算法

- 预处理：作边长 $\ell = \alpha r / \sqrt{d}$ 的网格划分，并作随机平移（平移量每维在 $[0, \ell]$ ）
- 找到所有使得 $\square \cap B(q, r) \neq \emptyset$ 的 $\square$ ，记这个 $\square$ 形成的集合是 Q
- 返回 $\sum_{\square \in Q} |\square \cap P|$ ，即与 Q 中 $\square$ 相交的数据点数，作为最终的估计
- 正确性： $B(q, r)$ 内的点都计入了；至多计入 $B(q, (1 + \alpha)r)$ 的点
- 复杂性：可以证明 $|Q| \leq 2^{d/\alpha^{2/3}} \text{poly}(d)$

作业十七： 高维Range Query

- 标程参数选取： $\ell = 3r$ ，近似比最坏是 $3\sqrt{d} + 1$

降维：
应对Curse of Dimensionality

第一类降维： JL Transform

概述

[Johnson-Lindenstrauss, 1984]; 简称JL

JL是一种一般的降维方法，保n个 $\mathbb{R}^d$ 上的数据点集 P 中任何两点之间的距离

给定误差 ϵ ，JL是一个映射 $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ ，这里 $m = O(\epsilon^{-2} \log n)$

- 使得： $\forall x, y \in P, \|f(x) - f(y)\|_2 \in (1 \pm \epsilon) \cdot \|x - y\|_2$; $O(dm)$ 时间可算 $f(x)$

m 决定了降维的维度，称为target dimension，是降维的最重要性能指标

相对误差 ϵ ，是“最好”的近似比
可直接用于很多与距离有关的问题
以得到改进的近似算法

- 输入的 d 可大可小，目标维度只与 $\log n$ 和 ϵ^{-1} 有关

尤其对于例如 $d = n$ 时特别有效

这说明：对于近似算法来说， $O(\log n)$ 维可以说是“不失一般性”

作业十八： JL实际效果实验报告

- 在实际数据集上测试JL算法的性能
- 按照课上讲的，我们需要测试最坏的相对误差
- 但同时，我们也测试平均误差，可以预计平均误差远小于最坏误差

Linear Regression求解公式

- 将输入 $(x_1, y_1), \dots, (x_n, y_n)$ 写成矩阵 $X \in \mathbb{R}^{n \times d}$ 第 i 行是 x_i , 向量 $y := (y_1, \dots, y_n)$
- 那么问题可以写作 $\arg \min_{w \in \mathbb{R}^d} \|Xw - y\|^2$
 - 这里 $(X^\top X)^{-1}$ 未必存在, 存在条件是 X 的 d 列线性独立, 即 d 个feature是独立的
- 一般公式: 最优的 $w^* = (X^\top X)^{-1} X^\top y$
 - 也是通过对每个 w_i 求导后令导数 = 0得到方程, 然后这是方程的解
- 什么复杂度? $X^\top X$ 需要 $O(nd^2)$, 是主要复杂度项 (另外, 求逆需要 $O(d^3)$)

加速方法：Subspace版本的JL

这是因为 Xw 中 w 是 d 维的

- 特殊性：需要考虑的 $v = Xw - y$ 虽然是 n 维，但rank/独立变量数/自由度只有 d
- 设 $\mathcal{U}$ 是 $\mathbb{R}^n$ 的一个 d 维的子空间

在我们应用中是所有 $w \in \mathbb{R}^d$ 在变换 Xw 下的集合，即 $\{Xw : w \in \mathbb{R}^d\}$ 可以（不失一般性）看作是 n 维空间只有前 d 个维度非0的子集

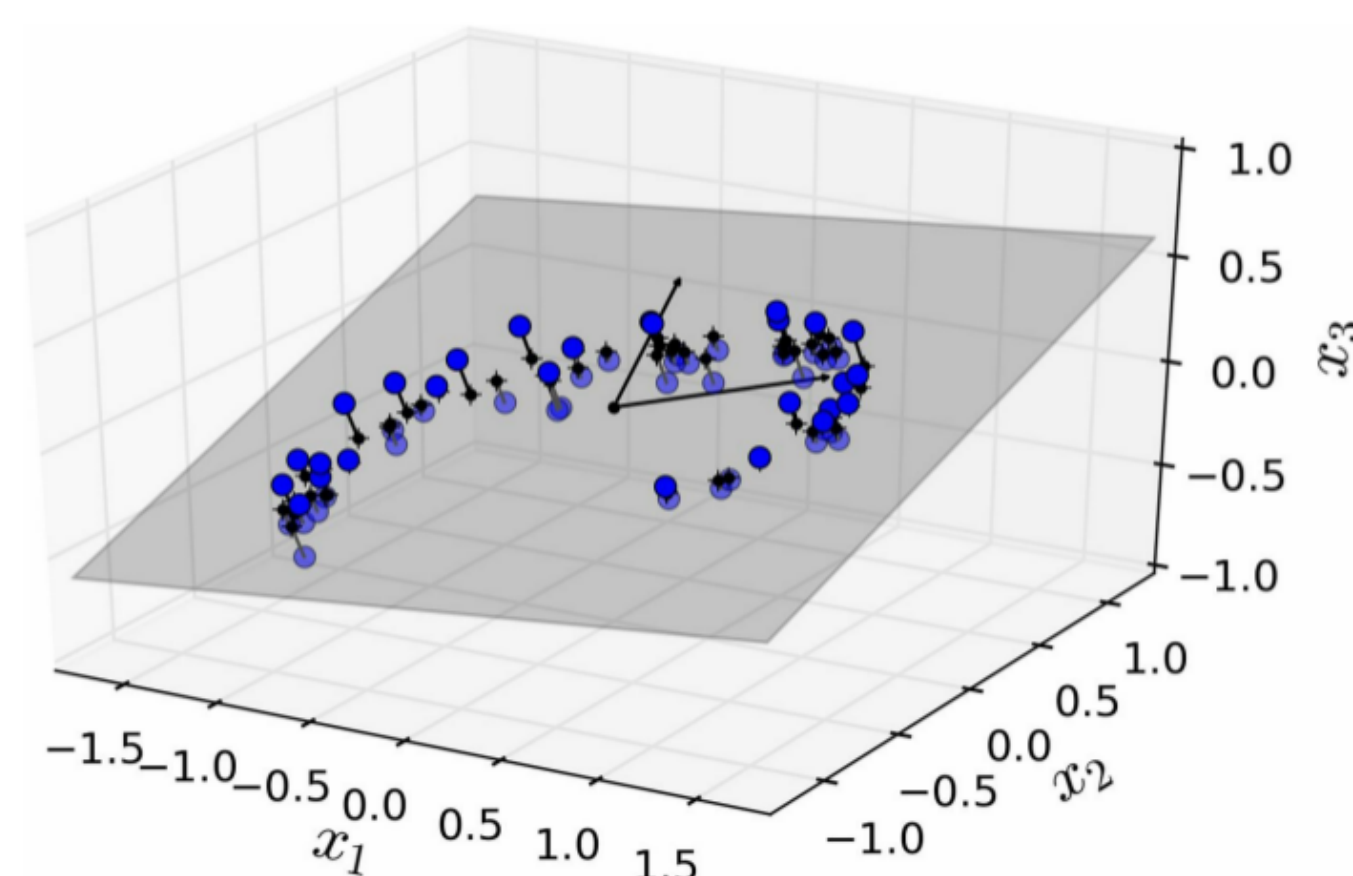
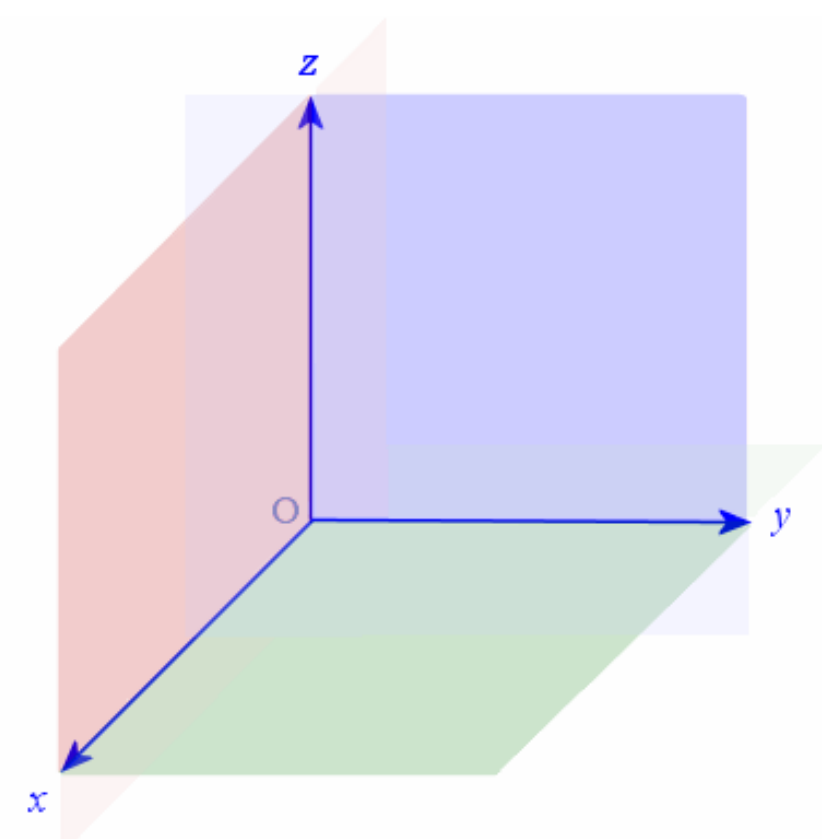


Figure 8-2. A 3D dataset lying close to a 2D subspace

例如 $\mathbb{R}^3$ 的x-y平面就是一个2维子空间，并且任意 $\mathbb{R}^3$ 上的2维子空间都可以通过旋转平移得到x-y平面

$Xw - y$ 可等价写作 $X'w'$ ： X' 和 w' 在 d 维基础上加一维

$$X' = \begin{bmatrix} X_{11}, \dots, X_{1d} & y_1 \\ \vdots & \ddots & \vdots & \vdots \\ X_{n1}, \dots, X_{nd} & y_n \end{bmatrix} \quad w' = \begin{bmatrix} w_1 \\ \vdots \\ w_d \\ -1 \end{bmatrix}$$

$O(\text{nnz}(X))$ 时间降维

设随机 $S \in \mathbb{R}^{m \times n}$ 每列只有一个均匀随机位置非0，且该元素取均匀随机 $\{-1, 1\}$

$$S = \begin{pmatrix} 0 & 0 & \cdots & 1 & 0 & -1 \\ \vdots & \vdots & \cdots & \vdots & \vdots & \vdots \\ 1 & \vdots & \cdots & \vdots & \vdots & \vdots \\ \vdots & 1 & \cdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & 0 & -1 & 0 \end{pmatrix}$$

下页解释 SX 可在 $O(\text{nnz}(X))$ 计算!

[Clarkson-Woodruff, STOC 13]

定理：用 $A' = S$ 替代 JL 矩阵 A ， $m = O(\epsilon^{-2} d^2 \cdot \text{poly log}(\epsilon^{-1} d))$ 有类似保证

$$\Pr[\forall v \in \mathcal{U}, \|Av\|^2 \in (1 \pm \epsilon) \cdot \|v\|^2] \geq 1 - \delta$$

d^2 会让最后降维后的矩阵维度稍大，后面会讨论

回忆 $\mathcal{U}$ 是 $\mathbb{R}^n$ 的 d 维子空间

高效求解linear regression：完整算法

- 根据刚刚提到的方法生成每列只有一个非0元素的 $m \times n$ 矩阵 S
- 预处理/计算 $A'X$ 和 $A'y$ ，这里 $A' = S$

m可以灵活选取，未必按照理论上界
但注意：需要 $m \geq d$
- 在 $\hat{X} = A'X$ 和 $\hat{y} = A'y$ 的新输入上利用 $w = (\hat{X}^\top \hat{X})^{-1} \hat{X}^\top \hat{y}$ 公式计算近似最优解 w
- 输出 w

需要采用高斯消元

作业十九：利用subspace版本JL高效求解regression

内存/缓存的locality影响效率

- 实现时注意矩阵相乘的循环顺序

```
for (i = 1; i <= d; i++)  
    for (j = 1; j <= d; j++)  
        for (k = 1; k <= m; k++)  
            xtx[i][j] += x[k][i] * x[k][j];  
for (i = 1; i <= d; i++)
```

- 一般的 $a[i][j] += b[i][k] * c[k][j]$ 的循环顺序应该是i k j

k应当放到最外层

第二类降维：PCA/SVD，目标函数

假设均值0且每列归一

输入 $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ ，求 orthonormal 的 m 个向量 $\mathbf{v}_1, \dots, \mathbf{v}_m \in \mathbb{R}^d$ ，最大化

$$\frac{1}{n} \sum_{i=1}^n \underbrace{\sum_{j=1}^m \langle \mathbf{x}_i, \mathbf{v}_j \rangle^2}_{\text{squared proj. length}}$$

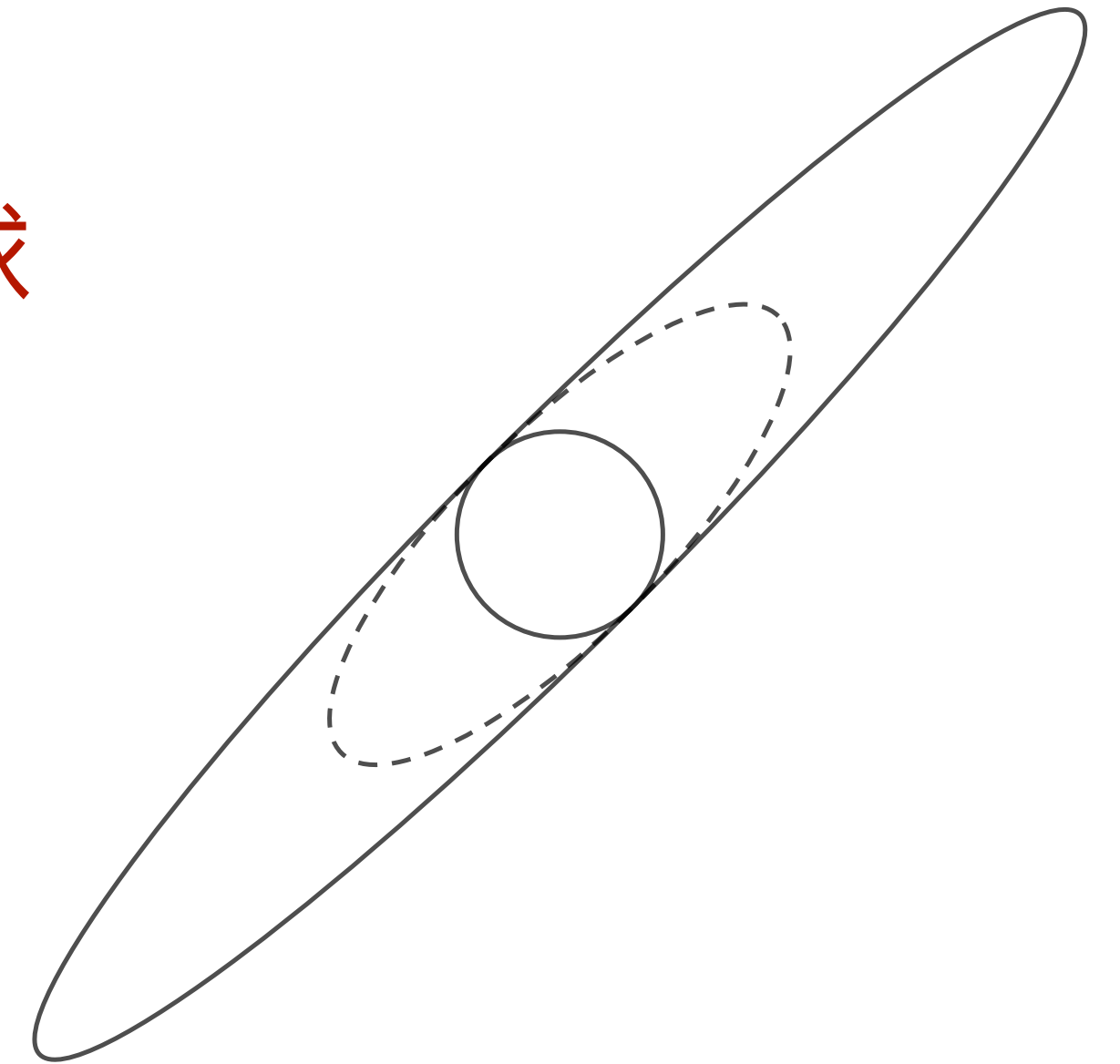
最优的这 m 个向量叫做 $\mathbf{x}_1, \dots, \mathbf{x}_n$ 的前 m 个 principle components

近似求Top Principle Component: Power Iteration

算法设计思路

- 回忆: $\mathbf{A} = \mathbf{X}^T \mathbf{X} = \mathbf{Q} \mathbf{D} \mathbf{Q}^T$ 本质上是将单位球映射到椭球
 - 椭球的最长轴就对应top principle component
- 任取一个向量 $\mathbf{u}$, 只要不与 $\mathbf{Qe}_1$ 垂直, 那么反复应用 $\mathbf{A}$ 后:
 - 将在 $\mathbf{Qe}_1$ 方向反复被拉伸
 - 若假设最长轴确实比其他轴长, 则最后 $\mathbf{u}$ 会非常贴近于 $\mathbf{Qe}_1$ 的方向

$\mathbf{Qe}_1$



如果看椭球, 则会在 $\mathbf{Qe}_1$ 上拉的非常长, 总体非常扁

算法

输入: $\mathbf{A} = \mathbf{X}^\top \mathbf{X}$

可以每一维独立 $\mathcal{N}(0,1)$ 后归一化生成

选取一个随机方向向量 $\mathbf{u}_0$

For $i = 1, 2, \dots$

$$\mathbf{u}_i := \mathbf{A}^i \mathbf{u}_0$$

根据实际情况设定“ $\approx$ ”的标准

如果 $\mathbf{u}_i / \|\mathbf{u}_i\| \approx \mathbf{u}_{i-1} / \|\mathbf{u}_{i-1}\|$ 则停止并返回 $\mathbf{u}_i / \|\mathbf{u}_i\|$

作业二十： 利用Power Iteration求Top PC

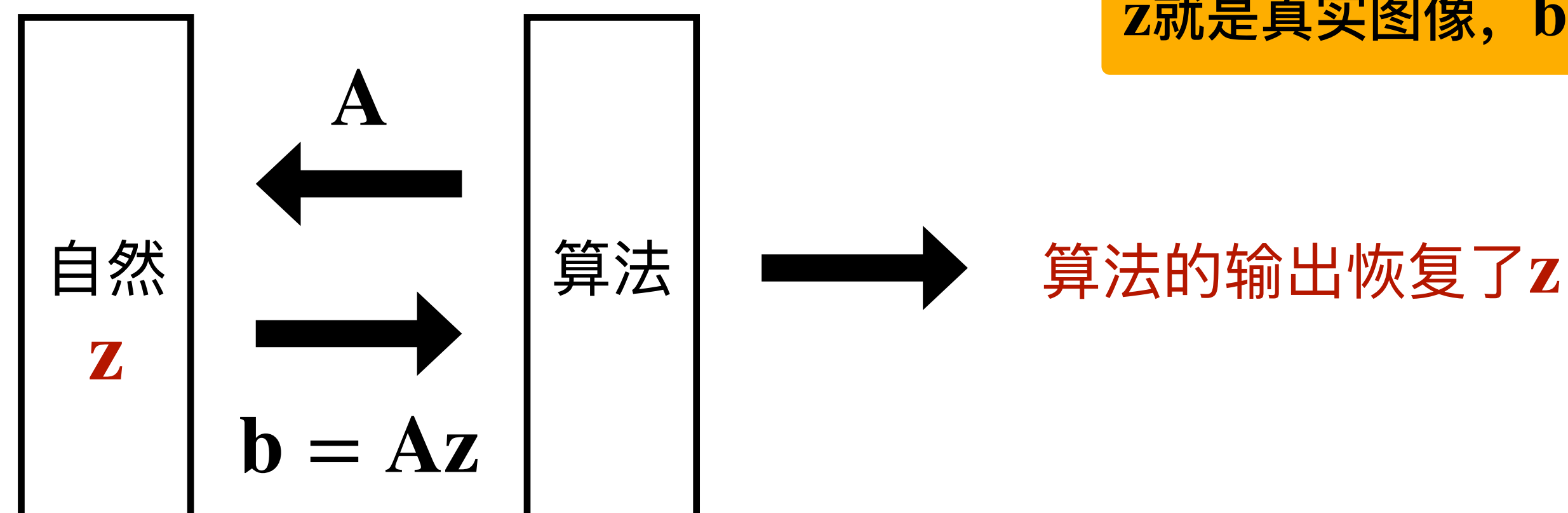
稀疏性

压缩感知具体设定

- 我们设计 m 个linear measurements（线性测量） $\mathbf{a}_1, \dots, \mathbf{a}_m \in \mathbb{R}^n$
- “自然”生成一个未知的信号 $\mathbf{z} \in \mathbb{R}^n$
- 某种硬件告诉我们信号在线性测量上的值 $\mathbf{b}$, $b_1 = \langle \mathbf{a}_1, \mathbf{z} \rangle, \dots, b_m = \langle \mathbf{a}_m, \mathbf{z} \rangle$
- 仅从 $\mathbf{b}$ 恢复 $\mathbf{z}$

目标是要同一组测量 $\mathbf{a}_1, \dots, \mathbf{a}_m$,
对“所有”可能的 $\mathbf{z}$ 都要有效恢复

想象设计一个影像传感器（摄像头）
 $\mathbf{z}$ 就是真实图像， $\mathbf{b}$ 就是传感器给出的测量值



压缩感知定理：精确版

定理：设 $\mathbf{A} \in \mathbb{R}^{m \times n}$ 其中 $m = O(k \log(n/k))$ 且 $\mathbf{A}$ 的每个元素都从 $\mathcal{N}(0,1)$ 独立采样。则 $\mathbf{A}$ 以高概率，同时对所有的 n 维的 k -稀疏的 $\mathbf{z}$ ，下面问题的唯一最优解恰好等于 $\mathbf{z}$ ：

$$\begin{aligned} \min \quad & \|\mathbf{x}\|_1 \\ \text{s.t.} \quad & \mathbf{Ax} = \mathbf{b} \end{aligned}$$

基于压缩感知的Sparse Recovery: 完整算法

- 预先给定的参数: k, n , 代表信号是 n 维的 k -稀疏的实向量
- 算法先生成一个 $m \times n$ 的随机高斯矩阵 $\mathbf{A}$, 其中 $m = O(k \log(n/k))$
- 算法得到 $\mathbf{b} \in \mathbb{R}^m$, 使得 $\mathbf{b} = \mathbf{A}\mathbf{z}$
- 算法继续计算右边LP的最优解 $\mathbf{x}$ 作为输出

每一维都是独立标准正态 $\mathcal{N}(0,1)$

压缩感知定理告诉我们大概率有 $\mathbf{x} = \mathbf{z}$

$$\begin{aligned} \min \quad & \sum_i y_i \\ \text{s.t.} \quad & \mathbf{Ax} = \mathbf{b} \\ & y_i - x_i \geq 0, \quad \forall i \\ & y_i + x_i \geq 0, \quad \forall i \end{aligned}$$

作业二十一： 压缩感知实验报告

作业需要调用一个解LP的算法，可以使用任何solver

Sparse Recovery与数据流

重要工具

技术上说，这里允许负频数，并且只要不是0频数，都算作support里面

- 复习：称一个频数向量 $\mathbf{x}$ 是 k -sparse的若 $\|\mathbf{x}\|_0 \leq k$
- Sparse recovery是一个数据流算法，给定一个参数 k ：

回答Yes/No

- 检测数据流（的频数向量）是否是 k -sparse的

- 如果Yes，就把 $\text{supp}(\mathbf{x})$ 完全恢复出来

即输出一个集合，等于 $\text{supp}(\mathbf{x})$ ，一共至多 k 个元素

结论：存在一个高概率成功的 $O(k \cdot \text{poly} \log n)$ 空间的sparse recovery算法

更新时间 $\text{poly} \log n$
查询时间 $O(k \cdot \text{poly} \log n)$

n 是domain大小，也就是频数向量的长度/维数

算法：利用Sparse Recovery解决数据流直径问题

- for $i = 0, \dots, O(\log(\Delta))$ 以2倍为步长穷举/猜测直径的值
 - 维护domain在 $\Delta \times \Delta$ 上的k-sparse recovery structure $\mathcal{S}_i$, 参数 $k = O(1/\epsilon)^d$
 - 当数据流插/删点 p 时, 找到 p 所在的 $\epsilon \cdot 2^i$ 格点中心 p' , 将 p' 从 $\mathcal{S}_i$ 插入/删除
- 数据流结束时, 询问每一个 $\mathcal{S}_i$ 是否k-sparse
- 找到*i*值最小的返回Yes的 $\mathcal{S}_i$, 返回恢复的点集并求该点集直径作为结果返回

刚刚看到, 猜测 $\geq \text{diam}(P)$ 时都会返回Yes
因此需要找符合条件的最小*i*值来找到 $\approx \text{diam}(P)$ 的猜测
事实上, 可能会找到更小的*i*, 但这只会让解更加精确

作业二十二：欧氏点集直径的数据流算法

**感谢同学们的支持和付出！
祝同学们学业进步、前程似锦！**

课程评估

- 请同学们花几分钟时间参与一下课程评估
- PC端可访问 kcpq.pku.edu.cn
- 手机可扫码关注“本科课程评估”
 - 首页—输入“学号”及“密码”登录—任务评价
 - 非本校同学： 点击个人设置—校外绑定—输入“学号”，“密码”默认为学号
 - 根据“我的任务”中的课程， 填写问卷并点击“提交”。

